

Dati leakage

Accesso da parte di utenti non autorizzati a dati privati

Due tipologie:

- **Explicit Information Flow**:
il dato privato viene sceduto direttamente
- **Implicit Information Flow**:
viene condiviso un dato non privato il cui valore dipende direttamente dal valore di uno privato

Necessità della definizione di un **secure information flow**, e di una sua certificazione

Un'istruzione condizionale causa l'inizio di un implicit flow;

Tutte le istruzioni eseguite a causa dell'istruzione condizionale dipendono da essa, e ciò vale anche per le funzioni

Multilevel security policy

Mechanism che permette la classificazione di dati e utenti in una **gerarchia di livelli di sicurezza**

Consiste dei seguenti elementi

- un set di **livelli di sicurezza** S
- una **relazione tra livelli di sicurezza** $\sqsubseteq$, che è riflessiva, transitiva e asimmetrica

Esempio:

$$\mathcal{L} = (S, \sqsubseteq), S = \{\text{public, private}\}$$

$$\sqsubseteq : \text{Public} \sqsubseteq \text{Private}$$

Propriets

- **Secure Information Flow**

Un output di livello inferiore non deve rivelare informazioni di un input di livello più alto.

- **Non interference**

Un utente di livello inferiore non può osservare/non deve essere influenzato da alcuna modifica nei livelli superiori.

Secure Information Flow

- **Typing approach**:

le informazioni sulla sicurezza di una variabile appartengono al suo tipo.

- **Semantic approach**

Si esegue il programma (complessità alta)

- **Abstract interpretation del semantic approach**

Si esegue il programma su un dominio astratto

Quest'ultimo è vantaggioso rispetto al primo perché è in grado di conservare informazioni sul comportamento dinamico del programma

Abstract interpretation of operational semantic per il Secure Flow

È un metodo per analizzare l'operational semantic di un programma, focalizzandosi sugli aspetti della sicurezza.

Non vengono considerati i reali valori che assumono i dati, preservandone invece le proprietà di sicurezza

Un programma è definito come: $P = \langle C, H, L \rangle$ dove

- $C \in \text{com}$, ovvero un comando
- H , le variabili ad alto livello di sicurezza
- L , le variabili a basso livello

$$H \cup L = \text{Var}(C), H \cap L = \emptyset$$

Enhanced Operational Semantics

Uno strumento che arricchisce l'operational semantics, permettendoci di:

- gestire valori (K, σ) assieme ai valori di sicurezza S
- eseguire istruzioni in un dato ambiente di sicurezza $\sigma \in S$

Notazione: $C(P, M)$, con $P = \langle C, H, L \rangle$ e $M(x) = (K, \sigma)$

Usato per scoprire violazioni di sicurezza

annotated value: durante l'esecuzione, σ definisce il limite superiore del livello di sicurezza.

execution environment σ può essere aumentato quando un'istruzione condizionale inizia.

Può essere diminuito quando i vari branch dell'istruzione condizionale terminano

$(c)^\sigma$ & $(d)^\sigma$: istruzioni e comandi vengono eseguiti sotto un certo ambiente di sicurezza σ

$\text{Mod}(c)$: istruzione che trova le variabili modificate dopo una sequenza di istruzioni

$\text{Imp}()$: aumenta il livello di sicurezza in caso di un implicit flow

Correctness

I-Security un programma è I-sicuro se il valore delle variabili a basso livello di sicurezza non cambia per ogni possibile valore delle variabili ad alto livello, quando il programma termina

Safe memory una memoria è sicura per un programma se ogni variabile "bassa" contiene un valore a basso livello di sicurezza in memoria

Teorema: L'enhanced semantics garantisce la I-security.

Abstract operational semantic

I sistemi generabili con l'enhanced semantic potrebbero essere infiniti, e non analizzabili staticamente.

L'abstract semantic approssima correttamente e in modo finito tutte le esecuzioni dell'enhanced semantic

Il primo passo è definire il dominio astratto, nel quale i nodi contengono un'astrazione degli stati

In questo contesto, vengono inoltre considerati solo i livelli di sicurezza invece della coppia (K, σ)

Ciò porta ad avere diversi path di esecuzione in caso di comandi condizionali e/o iterativi.

Dato una funzione astratta α :

- ne estraiamo il relativo σ
- applichiamo le stesse regole dell'enhanced nel dominio astratto
- applichiamo tutte le regole (true/false) in caso di istruzioni condizionali

Per verificare che un programma sia sicuro, costruiamo il sistema astratto e analizziamo tutti gli stati finali:

Un programma P è sicuro (has Secure Information Flow) se in ogni suo stato finale, ogni variabile x ha un valore $\tau \in \mathcal{T}$

Questo approccio si basa sui sistemi a stati finiti, e ha dunque il vantaggio di essere completamente automatico

Altre proprietà

Termination agreement: se non è possibile estrarre informazioni ad alto livello di sicurezza osservando la terminazione del programma

Timing agreement: se non è possibile estrarre informazioni ad alto livello di sicurezza osservando il numero di istruzioni eseguite

Information Flow:

Control Flow graph: grafo diretto (V, E) con V nodi ed E archi che indicano se un'istruzione è immediatamente eseguibile dopo un'altra

Postdomination: j postdominates i se i e j è in ogni path di esecuzione tra i e il nodo finale

IPD stack: mantenere informazioni sull'open implicit flow, viene aggiornato quando si entra/esce da un'istruzione di controllo

Lo stack di un programma può essere manipolato: la sua dimensione e il suo contenuto possono provocare un leak

Verifiche sullo stack: no overflow/underflow; no esecuzione a indirizzi indefiniti